Amrutam Telemedicine Backend: Architecture

Node.js 22 / Express - PostgreSQL 16 - Redis (optional) - Prometheus / Grafana / OpenTelemetry - Docker / Kubernetes / Terraform (AWS) | Repo: github.com/Ps493/Amrutam-telemedicine-backend

1. Overview and high-level architecture

A **modular monolith** (auth, doctors, consultations, prescriptions, payments, admin, audit; wired by a dependency-injection container) deployed as two processes: a stateless **API** and a **worker** for async jobs. Targets: **100k consultations/day** (~1.2 rps average, ~15 rps peak write; ~150 rps reads), **p95 < 200 ms reads / < 500 ms writes**, **99.95% availability**. Modules keep strict boundaries, so any can be extracted into a service without changing contracts.

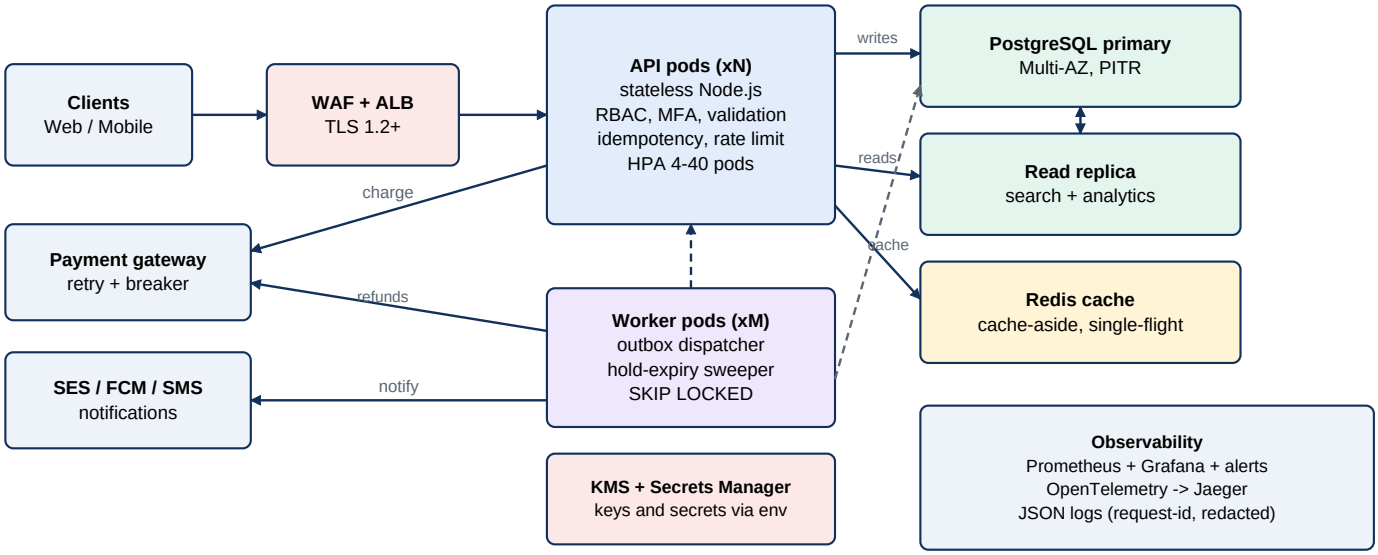


Figure 1. High-level architecture and data flow. Solid arrows = synchronous calls; dashed = asynchronous. The primary streams to the read replica, and WAL + snapshots are shipped to the DR region.

Request path: WAF/ALB, request-id, structured logging, helmet/CORS, body limit, rate limit, RED metrics, JWT authentication (HS256 pinned, iss/aud), RBAC permission check, MFA step-up for privileged roles, zod validation, idempotency, service, parameterised SQL, uniform error envelope.

2. Booking flow (sequence)

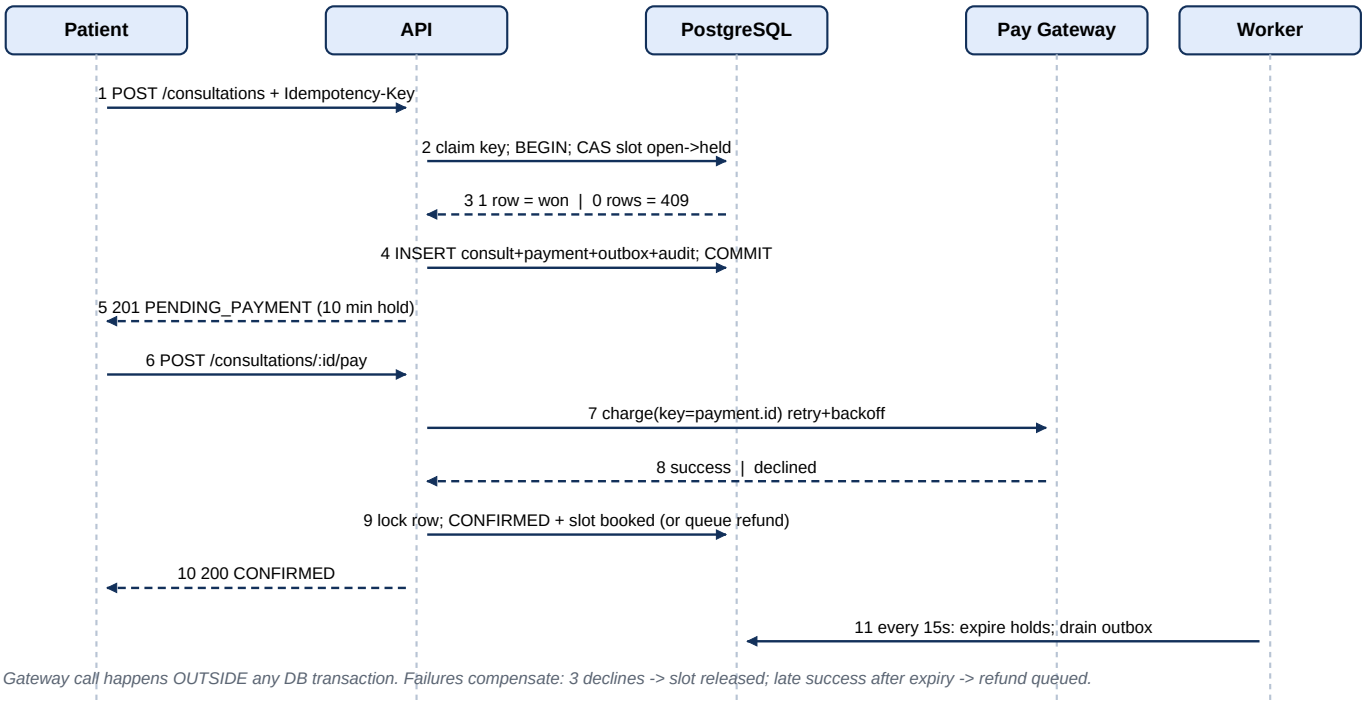
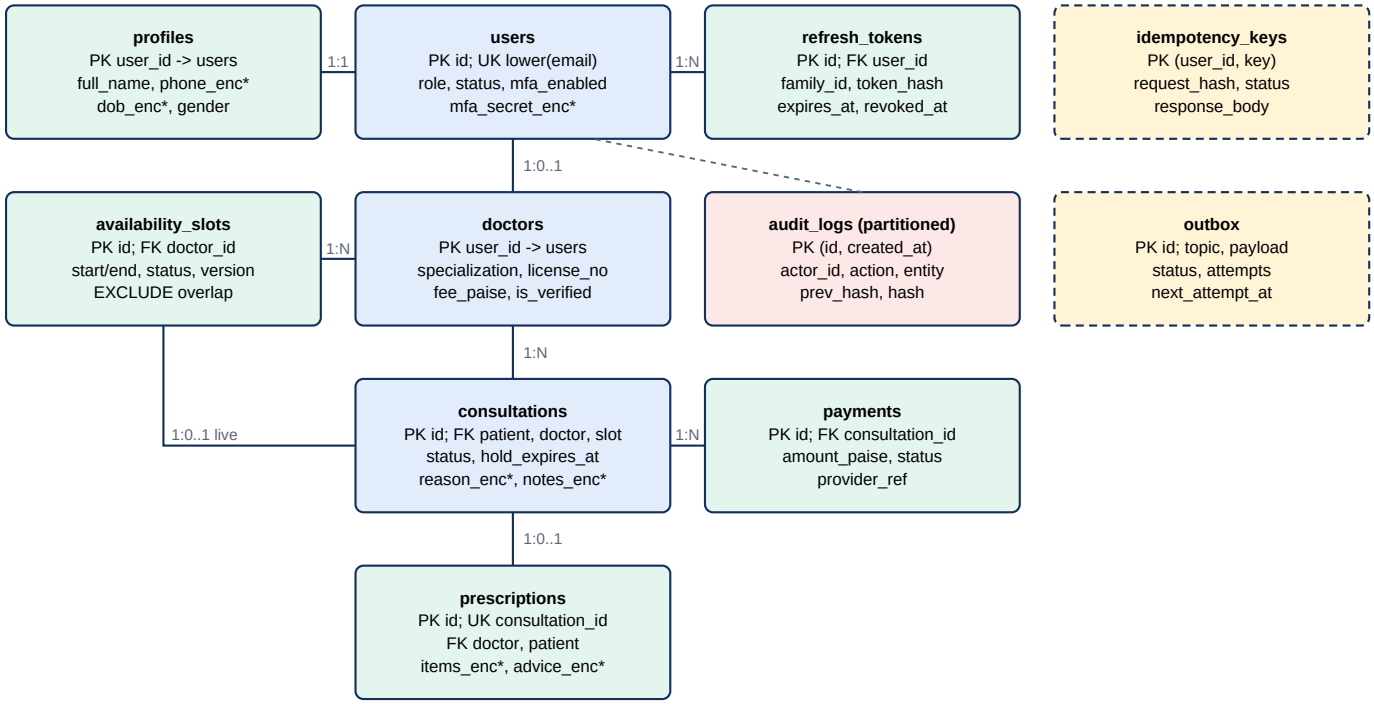


Figure 2. Booking and payment saga. The slot is reserved atomically first; payment is a separate, retryable step.

3. Data model (ER diagram)



** = AES-256-GCM encrypted at rest. Dashed boxes = infrastructure tables. consultations.patient_id -> users.id (patient).*

DB-enforced invariants: no overlapping slots per doctor; one live consultation per slot; one SUCCEEDED payment per consultation; audit_logs append-only. audit_logs.actor_id -> users.id (dashed, l

Figure 3. Entity-relationship diagram. Core tables: users, profiles, doctors, availability_slots, consultations, prescriptions, payments, audit_logs (plus refresh_tokens, idempotency_keys, outbox).

4. API schema

REST, versioned under **/v1**, fully described in **openapi.yaml** (25 paths; live Swagger UI at **/docs**). Uniform error envelope `{error:{code,message,details,requestId}}`. Pagination with hard caps. **Idempotency-Key header is required** on booking, payment, cancellation and prescription writes. Main groups: Auth (register, login, refresh, logout, MFA), Users, Doctors (search, slots), Consultations (book, pay, start, complete, cancel, prescription), Admin (analytics, audit logs, verify doctor), Ops (/healthz, /readyz, /metrics).

5. Retry and backoff strategy

Where	Policy	Why
Payment gateway	Retry only **transient** errors (timeout/5xx), 3 retries, exponential backoff with **full jitter**: rand(0, min(5 s, 100 ms x 2^n)). Provider-side idempotency key = payment.id. **Circuit breaker** opens after 8 consecutive failures (15 s cool-down).	No retry storms; never double-charge; fail fast (502) when provider is down
DB transactions	Retry on serialization failure (40001) and deadlock (40P01): 3 retries, 20 ms base.	Transaction bodies are idempotent units
Outbox events	Per-event backoff rand(0, min(10 min, 2 s x 2^n)); **dead-letter after 8 attempts**; alert on backlog.	At-least-once delivery; consumers are idempotent
Clients	Any write can be retried safely with the same Idempotency-Key.	Network ambiguity

6. Data partitioning and scaling

- **audit_logs** (implemented): RANGE partitions by month on created_at; worker pre-creates the next month; retention = detach, archive to S3 Object Lock (WORM), drop.
- **consultations / payments** (~36M rows/year): unpartitioned in year 1 with patient/doctor/time indexes; beyond ~200M rows move to RANGE(scheduled_start) monthly partitions.
- **availability_slots**: intentionally not partitioned, because the EXCLUDE overlap constraint is unsupported on partitioned tables; a partial index on status=open keeps the hot set small.
- **Reads:** search and analytics go to a read replica. **Beyond one primary:** shard by doctor_id hash (the contention key), CQRS read model for patient history.

7. Caching and concurrency handling

- **Cache-aside** for doctor search (30 s) and admin analytics (60 s), TTL jitter, **single-flight** to stop cache stampedes, prefix invalidation on slot/booking changes. A cache outage degrades to the DB, never to errors. Bookings and payments are never served from cache.
- **Double-booking prevention, three layers:** (1) atomic compare-and-set UPDATE ... WHERE status = open; (2) partial UNIQUE index on consultations(slot_id) for live statuses; (3) SELECT ... FOR UPDATE on state transitions. **Proven by test:** 12

concurrent patients race for one slot, exactly 1 gets 201 and 11 get 409.

- No DB transaction is held across a network call; the gateway charge happens between two short transactions.

8. Transaction management and sagas

Saga step	Local ACID transaction	Compensation
1 Reserve	slot held, consultation PENDING_PAYMENT, payment INITIATED, outbox, audit	Sweeper expires unpaid holds: slot reopened, consultation EXPIRED
2 Charge	External call with provider idempotency key (outside DB tx)	3 declined attempts: slot reopened, consultation CANCELLED
3 Confirm	payment SUCCEEDED, consultation CONFIRMED, slot booked (row lock)	Late success after expiry: refund queued via outbox
Cancel	slot reopened, consultation CANCELLED, outbox(refund)	Refund worker retries until payment REFUNDED

The **transactional outbox** publishes events only if the state change committed (no dual-write problem). Lifecycle changes are validated by a pure, unit-tested state machine. Every state change writes its audit entry inside the same transaction.

9. Backup, disaster recovery and availability

Item	Target	Mechanism
Availability	99.95% (~22 min/month)	Multi-AZ Postgres (auto-failover < 2 min), 4+ API pods over 3 AZs, PodDisruptionBudget, zero-downtime rolling deploys, readiness probes, graceful shutdown
RPO	<= 5 minutes	Continuous WAL / point-in-time recovery, 14-day retention, cross-region backup replication
RTO	<= 60 minutes (regional loss)	Terraform recreates the stack in the DR region, restore latest backup + WAL, health-checked DNS failover
Backups	Daily + WAL; weekly pg_dump	KMS-encrypted S3 with Object Lock; **monthly restore drill** verified with the audit-chain check
Keys	No data loss on rotation	Versioned keyring; rotate-keys script re-encrypts in batches (tested)

10. Observability and latency budget

Metrics: per-route latency histograms aligned to the 200/500 ms SLOs, bookings and payments by outcome, outbox backlog, DB pool waits, idempotent replays. **Alerts:** p95 SLO breaches, error-budget burn-rate for 99.95%, outbox backlog, pool saturation, payment-failure spike. **Logs:** JSON with request-id on every line and PII redaction. **Traces:** OpenTelemetry auto-instrumentation (http, express, pg, redis). **Budget:** reads are 1-3 indexed queries (~20-60 ms; cached ~5 ms); writes are one short transaction (~15-40 ms); gateway latency is isolated to /pay. **Measured locally (sandbox):** cached search p95 ~76 ms at 20 concurrent, uncached ~6 ms. **Known trade-off:** the audit hash-chain takes a global advisory lock at the end of each write transaction, which is fine at ~15 rps peak; at 10x scale use per-shard chains or batch-sealed Merkle roots.